Natural Language Processing (NLP)

Comprehensive Fundamental & Advanced Notes

A rigorous guide covering linguistic foundations, text preprocessing, vector space models, traditional machine learning, deep learning architectures, sequence models, transformers, and large language models.

Author: NLP Academic Series
Scope: Complete Theoretical & Practical Reference
Version: 2.6 (Expanded Edition)

Module 1: Introduction & Foundations of NLP

Natural Language Processing (NLP) sits at the intersection of computer science, artificial intelligence, and linguistics. It focuses on enabling computers to understand, interpret, and manipulate human language. Human language is inherently ambiguous, context-dependent, and evolving, making computational treatment extraordinarily challenging.

1.1 The Linguistic Hierarchy

To process human language computationally, NLP decomposes text into hierarchical linguistic layers:

- **Phonetics & Phonology:** Study of speech sounds and acoustic properties (relevant for speech-to-text and text-to-speech).
- **Morphology:** Study of word formation, roots, prefixes, and suffixes (e.g., decomposing "unbreakable" into *un-*, *break*, and *-able*).
- **Syntax:** Study of grammar, sentence structure, and the arrangement of words to form well-formed phrases.
- **Semantics:** Study of literal meaning of words, phrases, and sentences.
- **Pragmatics:** Study of meaning in context, speaker intent, and situational interpretation.

1.2 Core Challenges in NLP

Natural language exhibits several properties that complicate computational modeling:

- **Ambiguity:** Words and sentences often have multiple interpretations. Lexical ambiguity occurs when a word has multiple meanings (e.g., "bank" as a financial institution or riverbank). Syntactic ambiguity occurs when a sentence can be parsed in multiple ways (e.g., "I saw the man with the telescope").
- **Synonymy and Polysemy:** Different words can express the same concept (synonymy), or the same word can express vastly different concepts depending on context (polysemy).
- **Idioms and Metaphors:** Non-literal expressions where phrase meaning cannot be derived directly from the constituent words (e.g., "kick the bucket").
- **Discourse and Coreference:** Pronouns and references span across sentences, requiring cross-sentence tracking of entities.

> **Historical Evolution of NLP**
>
> NLP evolved through three major eras: **Empirical/Statistical NLP** (1990s–2010s) driven by probabilistic models and machine learning; **Deep Learning NLP** (2010s–2017) leveraging neural networks, recurrent architectures, and embeddings; and the **Transformer & LLM Era** (2017–Present) powered by self-attention mechanisms and massive pretraining.

Module 2: Text Preprocessing & Corpus Preparation

Raw text data is noisy, unstructured, and high-dimensional. Text preprocessing transforms raw strings into clean, structured tokens suitable for mathematical modeling.

2.1 Tokenization

Tokenization is the process of splitting a continuous text stream into smaller units called tokens (words, subwords, or characters).

- **Word Tokenization:** Splits text on whitespace and punctuation boundaries. Simple yet vulnerable to out-of-vocabulary (OOV) words and compounding.
- **Character Tokenization:** Treats every character as a token. Eliminates OOV issues but yields very long sequences and loses word-level semantic structure.
- **Subword Tokenization:** Balances word and character granularity. Algorithms like Byte-Pair Encoding (BPE), WordPiece, and Unigram break rare words into frequent subword components (e.g., `"unbelievable"` → `["un", "believ", "able"]`).

2.2 Normalization Techniques

Text normalization standardizes text variations into canonical forms:

- **Lowercasing:** Converting all characters to lowercase to treat "Apple" and "apple" as identical.
- **Stop Word Removal:** Filtering out extremely frequent, low-information words (e.g., "the", "is", "at", "which") using predefined stop-word lists.
- **Stemming:** Heuristic rule-based truncation of words to their root forms (e.g., Porter Stemmer converting "running", "runner", "runs" into "run"). Stemming can produce non-words.
- **Lemmatization:** Vocabulary- and morphological-driven reduction to dictionary base forms (lemmas) using part-of-speech tags (e.g., converting "better" into "good").

> $$\textit{Tokenization} \rightarrow \textit{Lowercasing} \rightarrow \textit{Stop-word Removal} \rightarrow \textit{Lemmatization}$$

Module 3: Traditional Vector Space Models

Machine learning algorithms require numerical inputs. Vector space models represent text documents as numeric vectors in high-dimensional spaces.

3.1 Bag-of-Words (BoW) Model

The Bag-of-Words model represents a text as an unordered collection of word counts, ignoring grammar and word order but maintaining multiplicity.

Given a vocabulary $V = \{w_1, w_2, \dots, w_D\}$, a document d is represented as a vector $\mathbf{x}_d \in \mathbb{R}^D$ where each dimension corresponds to the frequency of word w_i in d.

3.2 TF-IDF (Term Frequency-Inverse Document Frequency)

Raw term frequencies overweight common words like "the" or "and". TF-IDF scales term frequencies by inverse document frequency to highlight words that are frequent in a specific document but rare across the entire corpus.

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

$$IDF(t, D) = \log \left(\frac{|D|}{1 + |\{d \in D : t \in d\}|} \right)$$

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

3.3 Similarity Metrics in Vector Spaces

To compare documents, spatial distance or angular similarity is computed:

- **Cosine Similarity:** Measures the cosine of the angle between two vectors, invariant to document length.
- **Euclidean Distance:** Geometric straight-line distance in Euclidean space (sensitive to magnitude).

$$\text{Cosine}(x, y) = \frac{x \cdot y}{\|x\| \|y\|} = \frac{\sum_{i=1}^D x_i y_i}{\sqrt{\sum_{i=1}^D x_i^2} \sqrt{\sum_{i=1}^D y_i^2}}$$

Module 4: Language Modeling & N-Grams

A Statistical Language Model (LM) assigns a probability to a sequence of words $P(w_1, w_2, \dots, w_n)$.

4.1 The Chain Rule of Probability

Using the chain rule of probability, any joint probability can be decomposed into conditional probabilities:

$$P(w_1, w_2, \dots, w_n) = \prod_{k=1}^n P(w_k \mid w_1, w_2, \dots, w_{k-1})$$

4.2 The N-Gram Assumption

Conditioning on the entire history is computationally intractable. The Markov assumption simplifies this by assuming that the probability of a word depends only on the preceding $N-1$ words.

- **Unigram ($N=1$):** $P(w_i)$ (independent words).
- **Bigram ($N=2$):** $P(w_i \mid w_{i-1})$.
- **Trigram ($N=3$):** $P(w_i \mid w_{i-2}, w_{i-1})$.

> $$P_{MLE}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

4.3 Smoothing Techniques

Maximum Likelihood Estimation (MLE) assigns zero probability to unseen N-grams in the corpus. Smoothing redistributes probability mass to handle zero-count events:

- **Laplace (Add-One) Smoothing:** Adds 1 to all numerator counts and vocabulary size V to the denominator.
- **Good-Turing Smoothing:** Re-estimates counts of N-grams based on the frequency of N-grams that occur one fewer time.
- **Kneser-Ney Smoothing:** State-of-the-art traditional back-off smoothing utilizing continuation probabilities.

Module 5: Distributed Representations & Word Embeddings

Traditional sparse representations (one-hot encoding) fail to capture semantic similarity and suffer from the curse of dimensionality. Distributed representations encode semantic meaning in dense, low-dimensional vectors.

5.1 Word2Vec (Mikolov et al., 2013)

Word2Vec introduced two efficient neural network architectures for learning dense word embeddings from massive unannotated corpora:

- **Continuous Bag-of-Words (CBOW):** Predicts target word w_t given its context window $[w_{t-c}, \dots, w_{t+c}]$.
- **Skip-gram:** Predicts surrounding context words given the target word w_t. Works exceptionally well with infrequent words.

> **The Semantic Vector Space Property**
>
> Word embeddings capture linear semantic relationships. Famous vector arithmetic in embedding spaces demonstrates concept preservation:
> $$ ext{Vec}(ext{"King"}) - ext{Vec}(ext{"Man"}) + ext{Vec}(ext{"Woman"}) \approx ext{Vec}(ext{"Queen"}) $$

5.2 Global Vectors (GloVe) & FastText

- **GloVe:** Combines global matrix factorization (co-occurrence matrix) with local context window methods.
- **FastText:** Extends Word2Vec by representing words as bags of character n-grams, enabling robust embedding generation for out-of-vocabulary (OOV) and misspelled words.

Module 6: Sequence Labeling & Part-of-Speech Tagging

Sequence labeling assigns a categorical label to every token in an input sequence.

6.1 Part-of-Speech (POS) Tagging

POS tagging assigns grammatical categories (Noun, Verb, Adjective, Adverb, Determiner, etc.) to each token in context. Essential for syntactic parsing and disambiguation.

6.2 Hidden Markov Models (HMMs)

HMMs are probabilistic generative models for sequence labeling. They assume that hidden states (tags Y) generate observable symbols (words X).

- **Transition Probabilities:** $P(y_i \mid y_{i-1})$
- **Emission Probabilities:** $P(x_i \mid y_i)$

The Viterbi algorithm is used to decode the most probable sequence of hidden states given an observation sequence.

6.3 Conditional Random Fields (CRFs)

CRFs are discriminative undirected graphical models that overcome label bias and independence assumptions inherent in HMMs by modeling the conditional probability distribution $P(Y \mid X)$ directly.

Module 7: Syntax & Constituency Parsing

Parsing uncovers the hierarchical syntactic structure of sentences.

7.1 Context-Free Grammars (CFGs)

A CFG is defined by a 4-tuple $G = (V, \Sigma, R, S)$ where V is non-terminal symbols, Σ is terminal symbols, R is production rules, and S is the start symbol.

7.2 Constituency vs. Dependency Parsing

- **Constituency Parsing:** Breaks sentences into nested phrase-structure constituents (Noun Phrases, Verb Phrases).
- **Dependency Parsing:** Focuses on directed binary grammatical relations between words (e.g., `nsubj`, `dobj`).

Module 8: Recurrent Neural Networks (RNNs)

Feedforward networks cannot process variable-length sequential data. RNNs maintain a hidden state vector that evolves recurrently over time.

8.1 Mathematical Formulation of Vanilla RNN

$$h_t = anh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

$$\hat{y}_t = ext{softmax}(W_{hy} h_t + b_y)$$

8.2 Vanishing and Exploding Gradients

Backpropagation Through Time (BPTT) requires multiplying weight matrices across time steps. When sequences are long, gradients either decay exponentially to zero (vanishing gradient) or grow exponentially to infinity (exploding gradient), making vanilla RNNs ineffective for capturing long-range dependencies.

Module 9: Advanced Recurrent Networks (LSTMs & GRUs)

To mitigate vanishing gradients, gated architectures were introduced.

9.1 Long Short-Term Memory (LSTM)

LSTMs introduce a memory cell c_t and three multiplicative gating mechanisms:

- **Forget Gate (f_t):** Decides what information to discard from cell state.
- **Input Gate (i_t):** Decides what new information to store in cell state.
- **Output Gate (o_t):** Decides what information from cell state to output as hidden state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

9.2 Gated Recurrent Unit (GRU)

GRUs simplify LSTMs by merging the cell state and hidden state, utilizing two gates: the update gate and the reset gate.

Module 10: Sequence-to-Sequence (Seq2Seq) & Attention Mechanisms

Many NLP tasks (Machine Translation, Text Summarization) map an input sequence to an output sequence of arbitrary length.

10.1 Encoder-Decoder Architecture

An encoder RNN processes the input sequence into a fixed-length context vector. A decoder RNN generates the output sequence from this context vector. Bottleneck: Fixed-length vectors struggle with long sequences.

10.2 The Attention Mechanism (Bahdanau et al., 2014)

Attention allows the decoder to dynamically focus on different parts of the encoder's hidden states for every output step, bypassing the fixed-length bottleneck.

$$e_{ij} = \text{score}(s_{i-1}, h_j), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

$$c_i = \sum_j \alpha_{ij} h_j$$

Module 11: The Transformer Architecture

Introduced by Vaswani et al. (2017) in "Attention Is All You Need", Transformers discarded recurrence entirely, relying exclusively on self-attention mechanisms and parallel computation.

11.1 Scaled Dot-Product Attention

Given Queries Q, Keys K, and Values V, attention weights are computed via scaled dot-products:

$$ext{Attention}(Q, K, V) = ext{softmax}\left(rac{QK^T}{\sqrt{d_k}} ight) V$$

11.2 Multi-Head Attention

Multi-head attention projects queries, keys, and values into h different subspaces, allowing the model to jointly attend to information from different representation subspaces at different positions.

$$ext{MultiHead}(Q, K, V) = ext{Concat}(ext{head}_1, \dots, ext{head}_h)W^O$$
$$ext{where } ext{head}_i = ext{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

11.3 Positional Encoding

Because Transformers process tokens in parallel without recurrence or convolution, positional encodings (sine and cosine functions of varying frequencies) are added to input embeddings to preserve token order.

Module 12: Pretrained Language Models & LLMs

Modern NLP is dominated by large-scale Transformer models pretrained on massive web corpora via self-supervised objectives, then fine-tuned or prompted for downstream tasks.

12.1 BERT (Bidirectional Encoder Representations from Transformers)

BERT uses the Transformer encoder architecture and is pretrained using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). Excellent for understanding tasks.

12.2 GPT (Generative Pre-trained Transformer) Series

GPT models use autoregressive Transformer decoders trained on causal language modeling (predicting the next token). They excel at text generation, few-shot prompting, and reasoning.

12.3 Alignment, RLHF, and Modern Prompt Engineering

Modern Large Language Models (LLMs) undergo Instruction Tuning and Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO) to align outputs with human intent, safety, and helpfulness.

Model Family	Architecture Type	Primary Pretraining Objective	Best Suited For
BERT	Encoder-only	Masked Language Modeling (MLM)	Classification, NER, QA
GPT-4	Decoder-only	Causal Language Modeling	Generation, Chat, Reasoning

T5	Encoder-Decoder	Text-to-Text Span Corruption	Translation, Summarization

Module 13: Evaluation Metrics in NLP

Quantifying model performance depends heavily on the specific NLP task:

- **Classification Metrics:** Accuracy, Precision, Recall, F_1-Score, and ROC-AUC.
- **Machine Translation & Generation:** BLEU (Bilingual Evaluation Understudy), ROUGE (Recall-Oriented Understudy for Gisting Evaluation), and METEOR.
- **Perplexity:** Standard intrinsic evaluation metric for language models measuring how well a probability model predicts a sample.

Summary & Future Directions

NLP has progressed from hand-crafted syntactic rules to statistical models, deep learning, and multi-billion parameter foundation models. Current frontiers include multimodal models (vision + language + audio), reasoning agents, parameter-efficient fine-tuning (PEFT/LoRA), and efficient inference scaling.